

TryHackMe Room Writeup

OWASP Top 10 2025: IAAA Failures

A01 · A07 · A09 — Broken Access Control · Authentication Failures · Logging & Alerting Failures

Difficulty: Beginner | Duration: 30 min | Status: Completed (100%)

Date: 22 March 2026

Room Overview

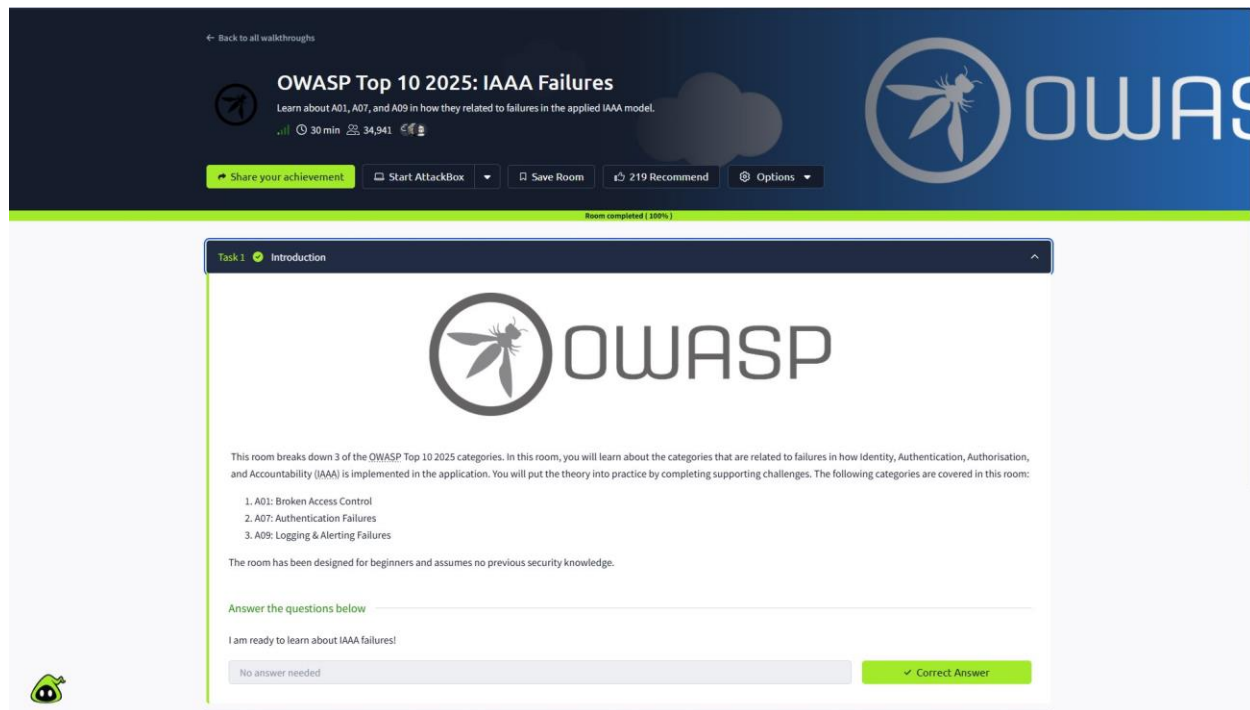


Figure 1 – TryHackMe room introduction page — Room completed 100%

This TryHackMe room covers three OWASP Top 10 2025 categories rooted in failures of the IAAA security model. Designed for beginners — no prior security knowledge required.

Categories covered:

- A01: Broken Access Control
- A07: Authentication Failures
- A09: Logging & Alerting Failures

Task 2 — What is IAAA?

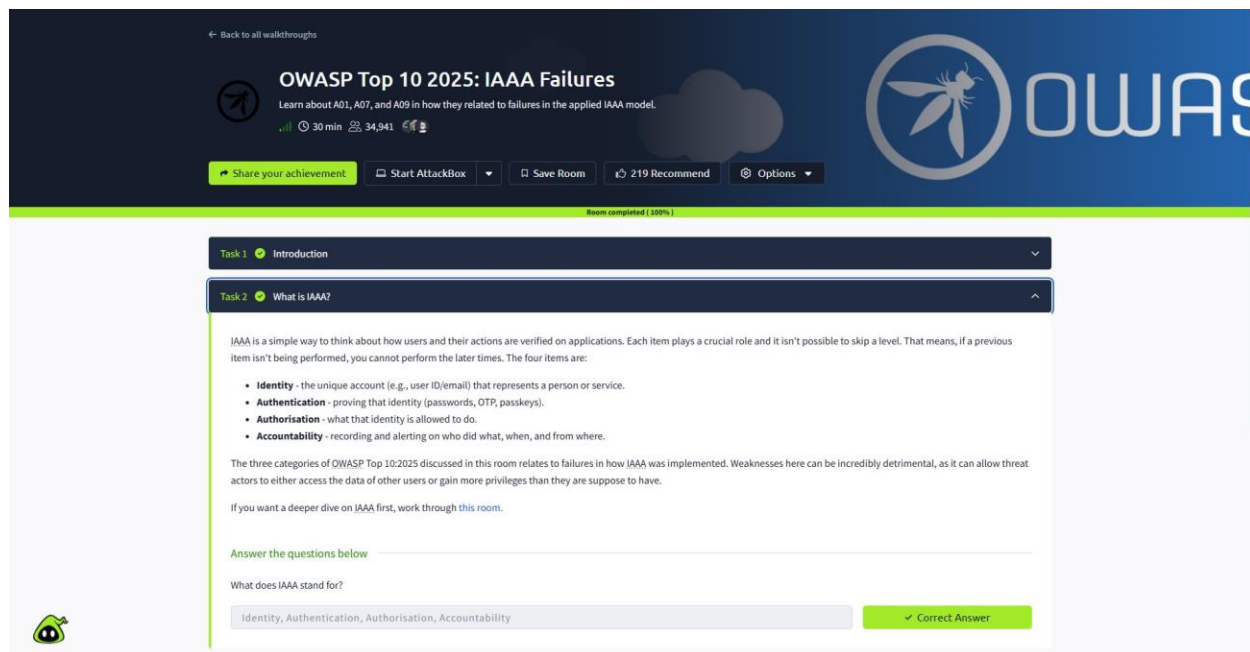


Figure 2 – Task 2: the four pillars of the IAAA model

Concept

IAAA is a layered model governing how users and their actions are verified. Each layer depends on the previous — you cannot skip ahead:

- **Identity** — the unique account (e.g. user ID / email) representing a person or service.
- **Authentication** — proving that identity (passwords, OTPs, passkeys).
- **Authorisation** — what that verified identity is permitted to do.
- **Accountability** — recording and alerting on who did what, when, and from where.

Answer

Q: What does IAAA stand for?

❑ **FLAG: Identity, Authentication, Authorisation, Accountability**

Task 3 — A01: Broken Access Control

Concept

Broken Access Control occurs when the server fails to enforce who can access what on every request. A classic example is IDOR (Insecure Direct Object Reference): by manipulating `?id=7` → `?id=6` in the URL an attacker can view or edit another user's data without any authorisation check.

This leads to two types of privilege escalation:

- **Horizontal** — same role, but reading a different user's data.
- **Vertical** — jumping to admin-only actions the attacker should not reach.

Challenge Walkthrough — Finding the Millionaire Account

❑ **Note:** Goal: Browse bank accounts on the static site by manipulating the id URL parameter and find the account holding more than \$1 million.

Step 1: Click "View Site" on Task 3. The banking site opens at:

```
https://bank.thm/accounts?id=1
```

Step 2: The id parameter controls which account is shown. There is no server-side ownership check, so any value can be entered freely.

Step 3: Change the id value in the URL bar from 1 up to 10 and press Enter each time:

```
https://bank.thm/accounts?id=2 https://bank.thm/accounts?id=3 ...  
https://bank.thm/accounts?id=10
```

Step 4: At id=7 the page reveals Account Balance: \$1,234,567.89. The account note contains the flag.

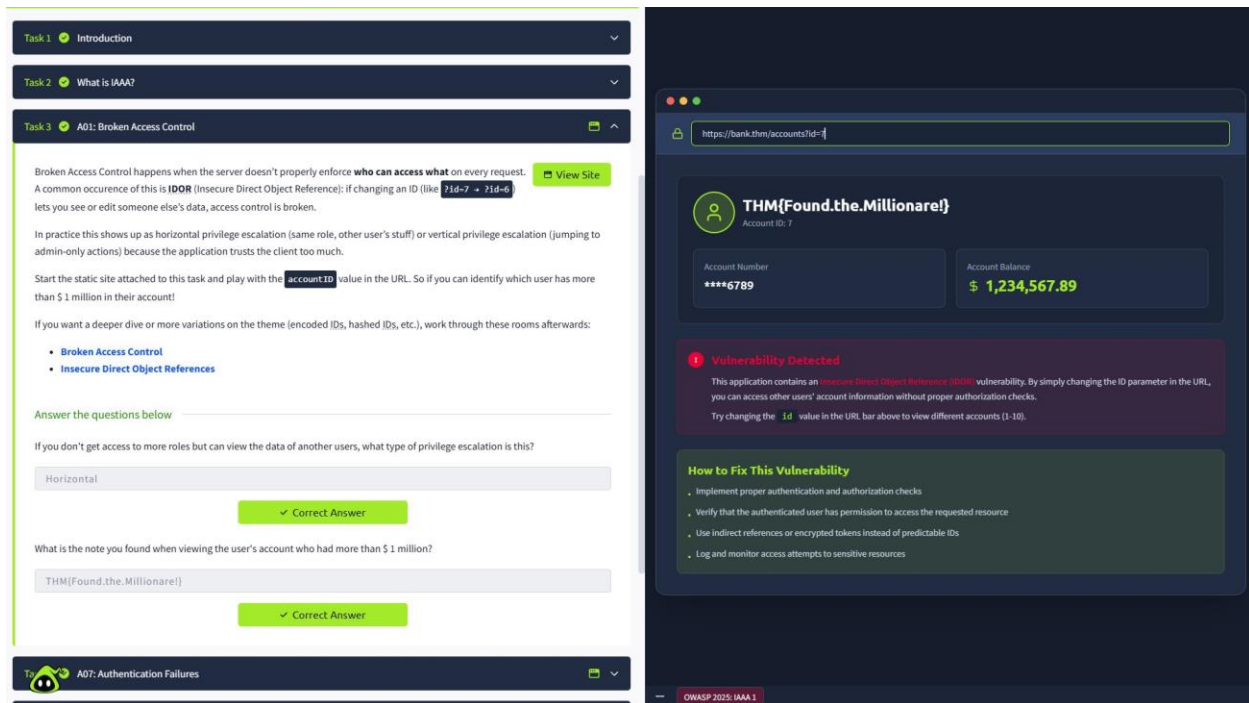


Figure 3 – id=7 reveals the millionaire account; the IDOR vulnerability banner explains the flaw; the flag appears in the account details

□ **Note:** Why it works: The application fetches the account matching the URL id without checking whether the logged-in user owns that account. Trusting client-supplied identifiers without server-side authorisation is the root of IDOR.

How to Fix This

- Verify on the server that the authenticated user owns the requested resource before returning any data.
- Replace sequential numeric IDs with non-predictable indirect references or GUIDs.
- Log and monitor all access attempts to sensitive account endpoints.

Answers

Q: If you can view another user's data without gaining extra roles, what type of escalation is this?

A: Horizontal

Q: What is the note found on the account with more than \$1 million?

□ **FLAG:** THM{Found.the.Millionare!}

Task 4 — A07: Authentication Failures

Concept

Authentication Failures occur when an application cannot reliably verify or bind a user's identity. Common weaknesses:

- Username enumeration — the app leaks whether a username exists.
- Weak or guessable passwords with no lockout or rate limiting.
- Logic flaws in the login / registration flow.
- Insecure session or cookie handling.

Challenge Walkthrough — Account Confusion Attack

Note: Goal: Access the admin dashboard by exploiting a case-sensitivity flaw in the registration flow. The real admin username is "admin" (lowercase).

Step 1: Click "View Site" on Task 4 to open bank.thm/register.

Step 2: The app checks username uniqueness in a case-sensitive way during registration, but normalises (lowercases) it during login. This mismatch is the vulnerability.

Step 3: Register a new account using the mixed-case username aDmiN with any password:

Username: aDmiN Password: anypassword Confirm Password: anypassword

Step 4: Click Register. The app accepts it — it sees "aDmiN" as a brand-new username.

Step 5: Log in with the credentials you just created (aDmiN / your password). The login flow lowercases the username, resolves it to the existing admin account, and grants you that session.

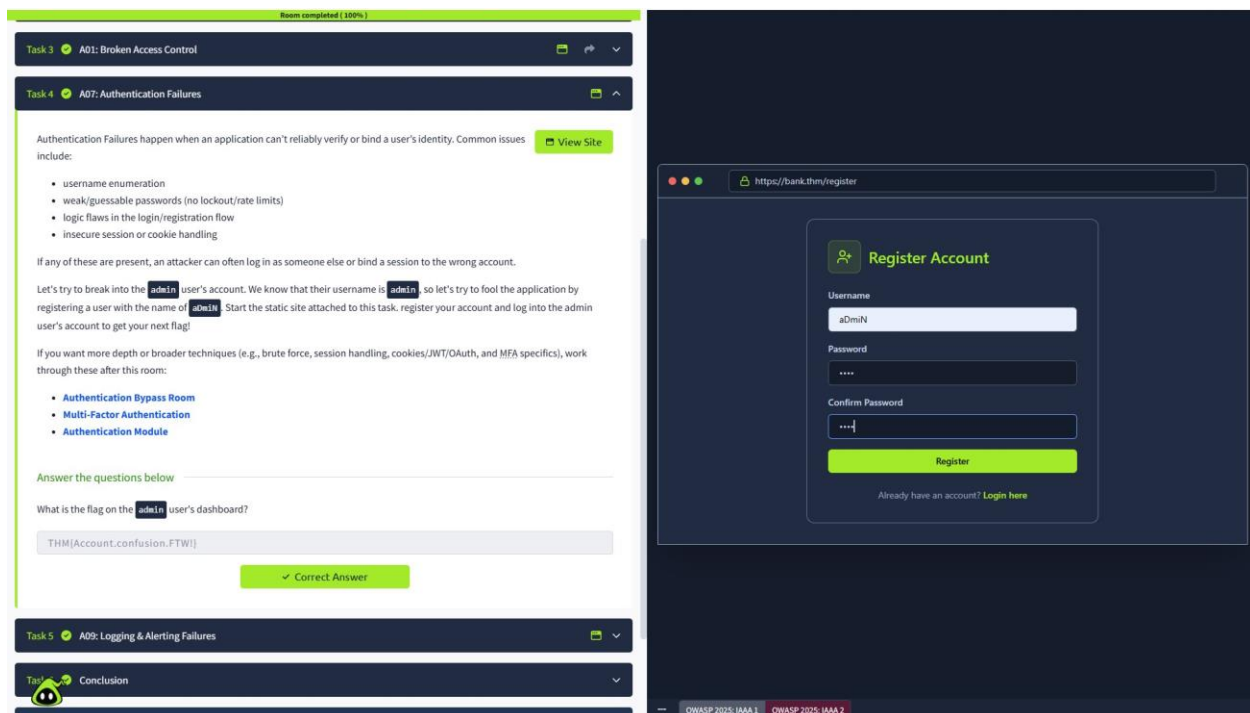


Figure 4 – Registering "aDmiN" on bank.thm/register to trigger the case-collision flaw

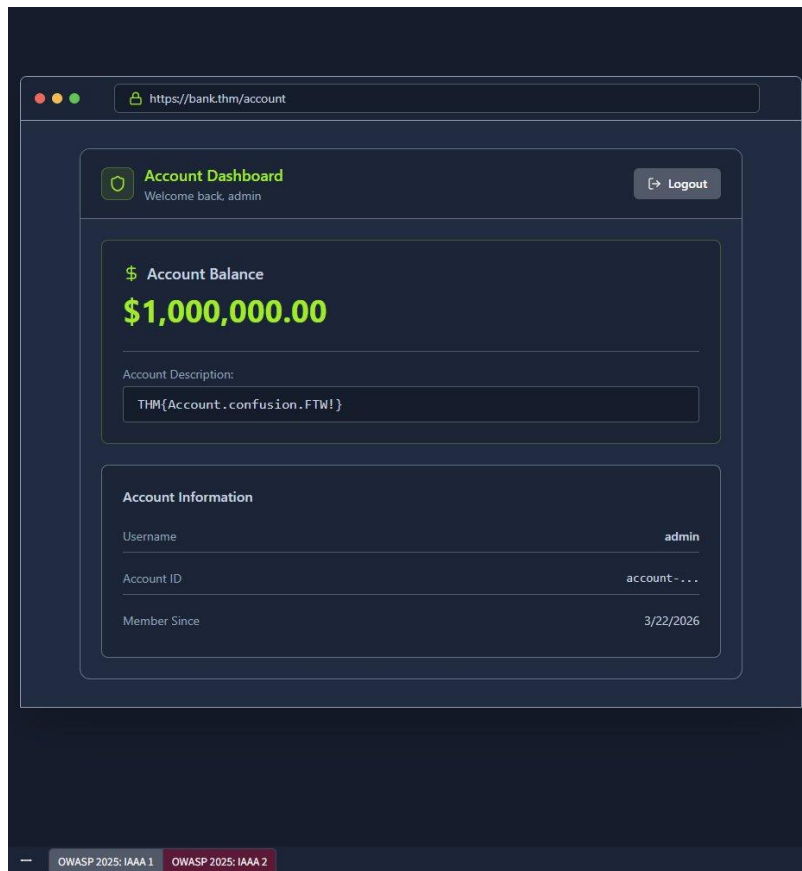


Figure 5 – Admin dashboard: "Welcome back, admin" — account confusion succeeded; flag visible in Account Description

□ **Note:** Why it works: Registration stores "aDmiN" as a new identity. Login normalises it to "admin" and binds the session to the pre-existing admin account. The inconsistency between the two flows lets an attacker hijack a privileged account without ever knowing its password.

How to Fix This

- Normalise (lowercase) usernames at the point of registration before uniqueness checks.
- Enforce a unique database index on the normalised username column.
- Rate-limit and lock out repeated failed login attempts.
- Rotate session tokens on any privilege change.

Answer

Q: What is the flag on the admin user's dashboard?

□ **FLAG:** `THM{Account.confusion.FTW!}`

Task 5 — A09: Logging & Alerting Failures

Concept

When applications fail to record or alert on security-relevant events, defenders cannot detect or investigate attacks. This breaks Accountability. Typical failures include:

- Missing authentication events (failed logins, lockouts, password changes).

- Vague error logs with no identifying context.
- No alerting on brute-force bursts or privilege changes.
- Short retention, or logs stored where attackers can tamper with them.

Challenge Walkthrough — SIEM Log Investigation

□ **Note:** Goal: Open the SIEM dashboard, analyse 40 log entries, and answer three investigation questions — attacker IP, compromised account, and endpoint accessed post-breach.

Step 1: Click "View Site" on Task 5 to open the SIEM Dashboard for bank.thm.

Task 5: A09: Logging & Alerting Failures

When applications don't record or alert on security-relevant events, defenders can't detect or investigate attacks. Good logging underpins **accountability** (being able to prove who did what, when, and from where). In practice, failures look like missing authentication events, vague error logs, no alerting on brute-force or privilege changes, short retention, or logs stored where attackers can tamper with them.

Let's take a look at what is required to perform an investigation of an application under attack. Start the static site attached to this task, perform your investigation, and answer the questions below. Then, think about how hard it would be to understand what happened during this attack if key pieces of this log information are missing.

If you want a deeper dive into logging for accountability, take a look at [this room](#).

Answer the questions below

It looks like an attacker tried to perform a brute-force attack, what is the IP of the attacker?

203.0.113.45

✓ Correct Answer

Looks like they were able to gain access to an account! What is the username associated with that account?

admin

✓ Correct Answer

What action did the attacker try to do with the account? List the endpoint accessed.

/supersecretadminstuff

✓ Correct Answer

SIEM Dashboard

Application logs for **bank.thm**

Total Logs: **40** | Suspicious Activity: **7** | Breached: **2**

DATE/TIME	STATUS	IP ORIGIN	METHOD	REQUEST URL
2025-01-15 08:23:14	200 OK	45.142.212.61	GET	/dashboard
2025-01-15 08:24:13	200 OK	198.51.100.42	GET	/accounts
2025-01-15 08:25:14	302 Redirect	87.128.45.78	GET	/oldpage
2025-01-15 08:27:03	200 OK	184.21.234.89	GET	/profile
2025-01-15 08:28:13	401 Unauthorized	51.195.67.123	POST	/login?user=john&pass=wrong
2025-01-15 08:29:42	200 OK	45.142.212.61	POST	/transfer
2025-01-15 08:30:01	200 OK	185.228.181.56	GET	/help
2025-01-15 08:33:28	200 OK	198.51.100.42	GET	/transactions
2025-01-15 08:33:33	302 Redirect	93.184.216.34	GET	/legacy
2025-01-15 08:35:14	200 OK	74.125.224.23	GET	/balance
2025-01-15 08:36:17	200 OK	208.88.152.45	GET	/cards
2025-01-15 08:38:00	401 Unauthorized	151.88.31.167	POST	/login?user=sarah&pass=test123

OWASP 2025: IAA 1 | OWASP 2025: IAA 2 | OWASP 2025: IAA 3

Figure 6 – SIEM Dashboard: 40 total logs · 7 suspicious activity flags · 2 breached accounts

Step 2: Scroll through the log table and look for a cluster of rapid 401 Unauthorized POST requests to /login from the same IP — this is the brute-force signature.

Step 3: Around 08:42 you will find six consecutive login attempts from IP 203.0.113.45 targeting the admin account:

2025-01-15 08:39:23	200 OK	172.217.22.67	GET	/loans
2025-01-15 08:40:46	200 OK	23.211.108.78	GET	/support
2025-01-15 08:41:12	200 OK	54.239.28.85	POST	/deposit
2025-01-15 08:41:58	302 Redirect	142.250.185.99	GET	/mobile
2025-01-15 08:42:09	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=admin123
2025-01-15 08:42:13	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=password
2025-01-15 08:42:17	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=12345
2025-01-15 08:42:21	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=letmein
2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=qwerty
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login?user=admin&pass=Password123
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadminstuff
2025-01-15 08:43:22	200 OK	13.107.42.14	GET	/statements
2025-01-15 08:44:15	200 OK	45.142.212.61	GET	/dashboard
2025-01-15 08:45:38	302 Redirect	104.244.42.193	GET	/api/v1
2025-01-15 08:46:11	401 Unauthorized	37.187.129.166	POST	/login?user=robert&pass=myspass
2025-01-15 08:47:01	200 OK	162.159.135.42	POST	/withdrawal

Figure 7 – Brute-force sequence from 203.0.113.45 — five failures followed by a 200 OK

1. /login?user=admin&pass=admin123 → 401 Unauthorized
2. /login?user=admin&pass=password → 401 Unauthorized
3. /login?user=admin&pass=12345 → 401 Unauthorized
4. /login?user=admin&pass=letmein → 401 Unauthorized
5. /login?user=admin&pass=qwerty → 401 Unauthorized
6. /login?user=admin&pass=Password123 → 200 OK ✓ Login succeeded!

Step 4: Immediately after the 200 OK the same IP makes a GET request to a sensitive admin endpoint:

GET /supersecretadminstuff → 200 OK

□ **Note:** Key insight: No rate limiting or lockout was in place. The attacker succeeded with six attempts in under a minute — entirely undetected until the SIEM was reviewed after the fact. Alerting on repeated 401s would have stopped this in real time.

Answers

Q: What is the IP of the brute-force attacker?

A: 203.0.113.45

Q: What username was associated with the compromised account?

A: admin

Q: What endpoint did the attacker access after gaining entry?

A: /supersecretadminstuff

Task 6 — Conclusion

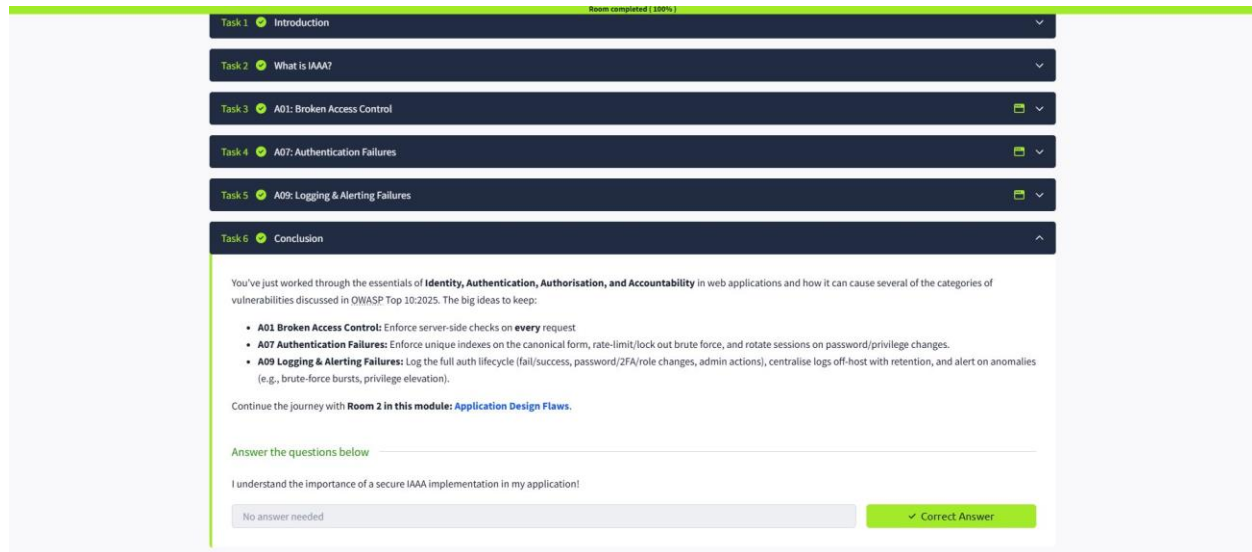


Figure 8 – All 6 tasks completed at 100%

This room showed how IAAA failures cascade into some of the most impactful web vulnerabilities. The three core takeaways:

- **A01 Broken Access Control:** Enforce server-side authorisation on every request. Never trust client-supplied resource identifiers without verifying ownership.
- **A07 Authentication Failures:** Normalise usernames to a canonical form before uniqueness checks. Rate-limit and lock out brute-force attempts. Rotate sessions on privilege changes.
- **A09 Logging & Alerting Failures:** Log the full authentication lifecycle. Centralise logs off-host with retention. Alert in real time on anomalies such as brute-force bursts and privilege elevation.

Room completed at 100%. All flags captured successfully.